

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

Ciclo Formativo de Grado Superior — DAM

ACTIVIDAD - 9.6

Alumno/a	David Guelar Franch
Curso	2ºDAM DIURNO BILINGÜE
Enlace GitHub	https://github.com/DGuelar/Practicas/tree/main/T9_Moviles

1. Enunciado de la actividad

Sigue el tutorial para desarrollar la actividad 9.6.

Importante: Personaliza tu desarrollo y que esa personalización se muestre en la ejecución del programa.

La actividad principal se llamará **MainActividad96** y el layout principal se llamará **layout_actividad96**.

Objetivo: Trabajar con `ScheduledThreadPoolExecutor`. Construiremos un programador de tareas que pondrá en marcha cuatro tareas en diferentes hilos.

2. Desarrollo de la actividad

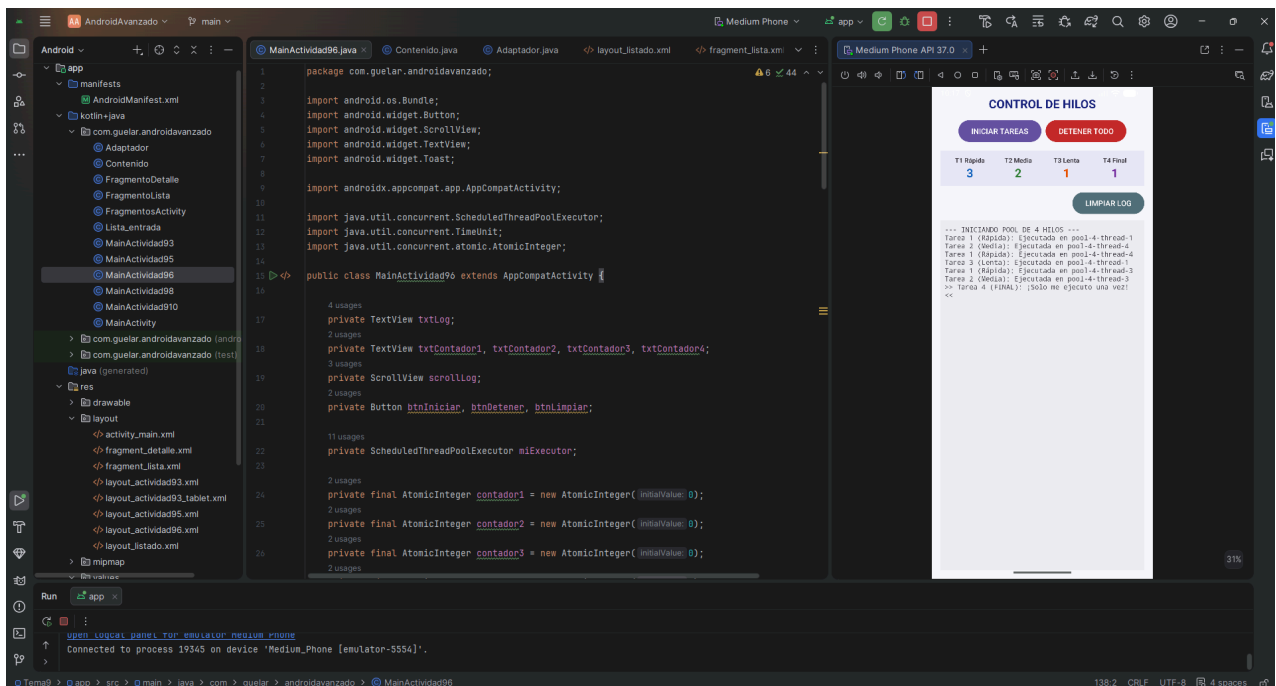
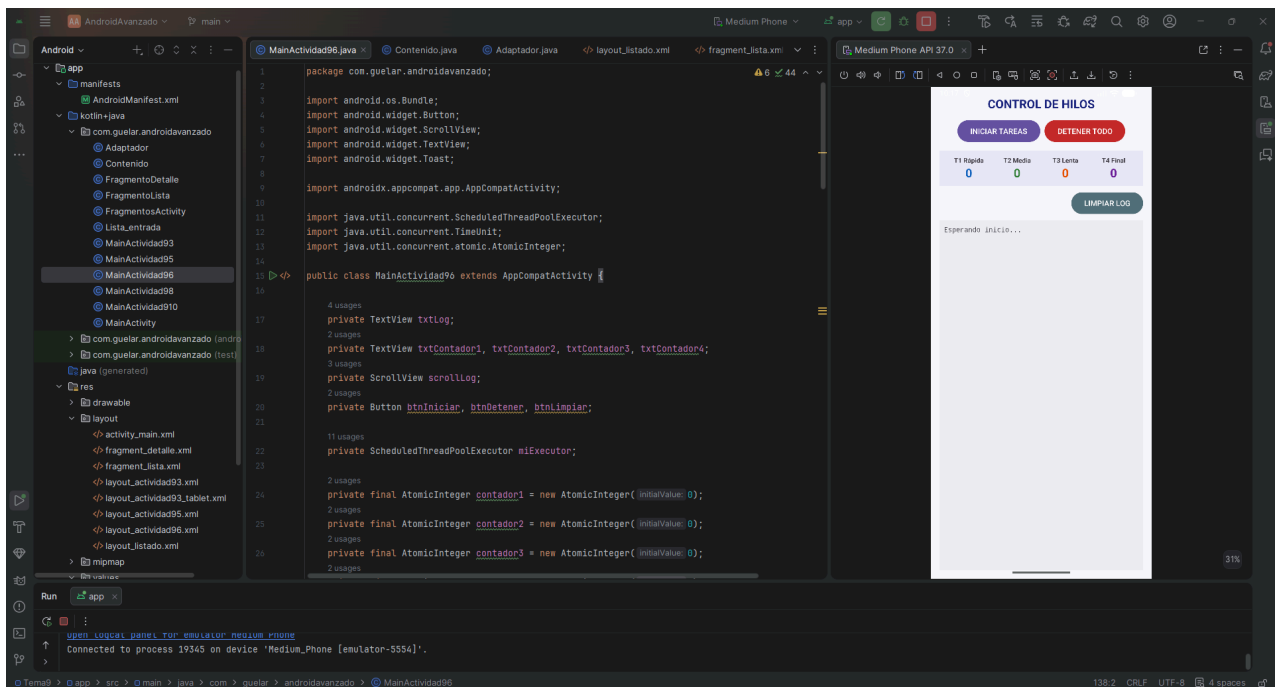
Un `ScheduledThreadPoolExecutor` es una clase de Java que implementa un grupo (pool) de hilos reutilizables con capacidad de temporización. A diferencia de crear un `Thread` nuevo para cada tarea (costoso en recursos), el pool mantiene un número fijo de hilos activos que se van repartiendo las tareas según van quedando libres. El número entre paréntesis del constructor indica cuántos hilos pueden ejecutarse simultáneamente.

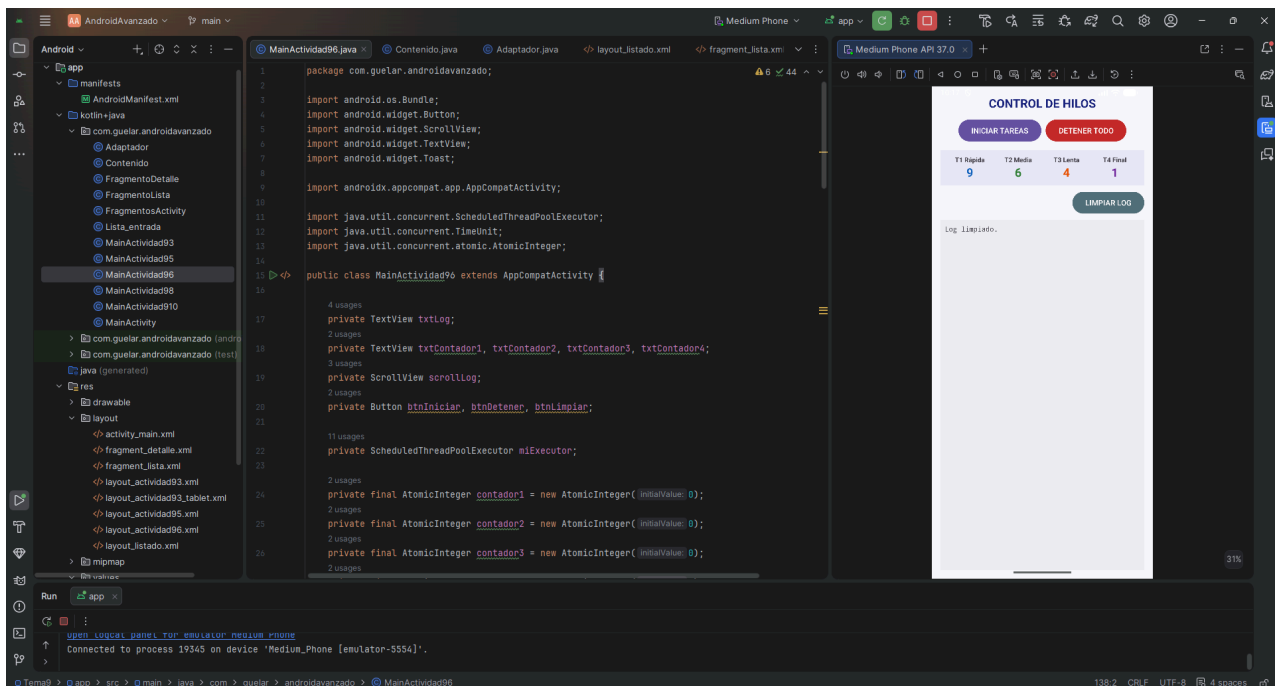
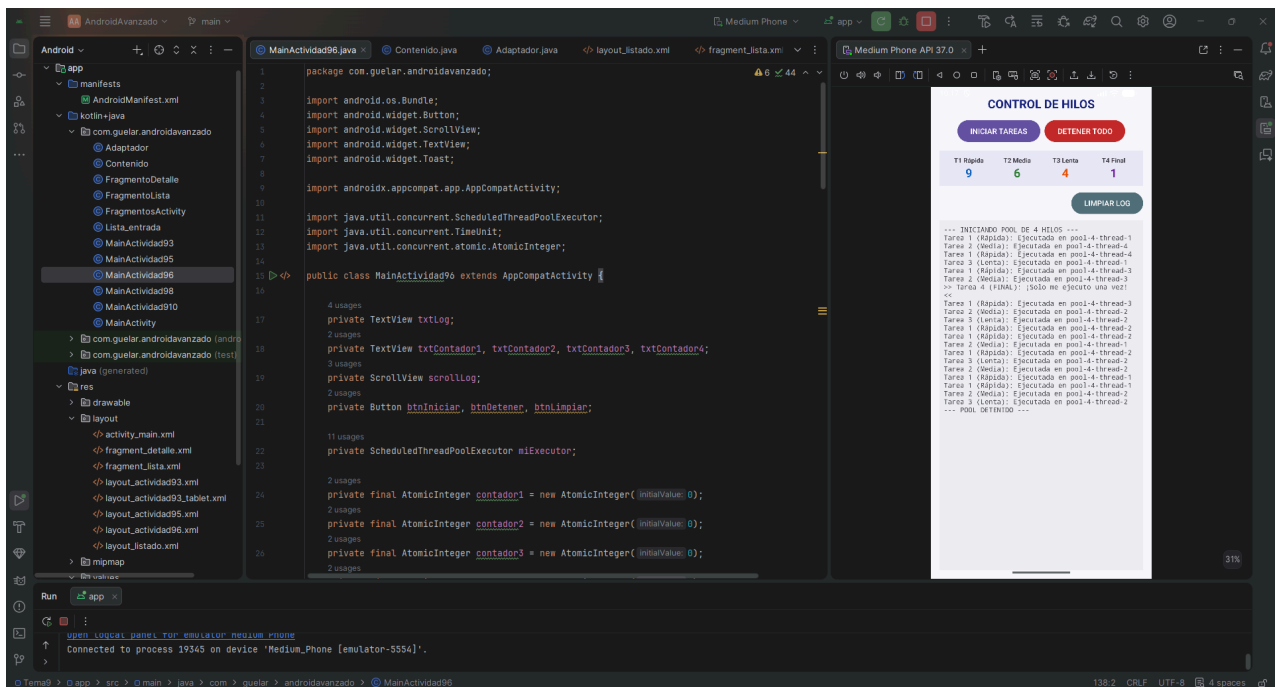
scheduleAtFixedRate vs schedule: `scheduleAtFixedRate(tarea, retardoInicial, periodo, unidad)` ejecuta la tarea indefinidamente cada X tiempo desde el inicio de la ejecución anterior. `schedule(tarea, retardo, unidad)` la ejecuta una única vez tras el retardo indicado. En este ejercicio la Tarea 4 usa `schedule` porque el enunciado pide que solo se ejecute una vez.

El problema del hilo de UI y `runOnUiThread`: Las tareas del `Executor` corren en hilos secundarios. Android no permite modificar vistas desde un hilo que no sea el principal. Por eso usamos `runOnUiThread()`, que encola el bloque de código en el hilo principal para que se ejecute de forma segura.

`AtomicInteger` para contadores en hilos concurrentes: Cuando varios hilos incrementan una variable al mismo tiempo, pueden producirse condiciones de carrera (dos hilos leen el valor, lo incrementan y escriben el mismo resultado, perdiendo una actualización). `AtomicInteger` garantiza que el incremento es una operación atómica, indivisible, sin posibilidad de interferencia entre hilos.

3. Capturas de pantalla





4. Referencias bibliográficas

- Documentación y temario impartido por el profesor
- Claude

RÚBRICA DE EVALUACIÓN

Criterios que tiene en cuenta el profesor para puntuar esta práctica:

CRITERIO	PUNTUACIÓN MÁXIMA	DESCRIPCIÓN
Solución al problema	6 puntos	0 = no resuelta / 6 = resuelta correctamente
Formato PDF	0,5 puntos	0 = no es PDF / 0,5 = formato correcto
Presentación y referencias	1 punto	0 = mala presentación / 1 = excelente con referencias bibliográficas
Expresión lingüística	1,5 puntos	0 = faltas y mala expresión / 1,5 = correcto, propio y sin faltas
Información adicional	1 punto	Aporta valor extra relevante al tema, además de todo lo anterior